

Advanced Certificate Programme in Agentic AI & RAG Engineering

Prompting + Evaluation Literacy + Capstone Kickoff
Phase 1 · Foundations, Eval Literacy & System Design



Today's outline

Day 1 · The mental shift

Concepts

What you'll learn

Three prompting patterns	20 min
Why unit tests don't work	15 min
Building a golden dataset	20 min

Live demos

Watch it built

Three responses, same prompt	10 min
Three EKA golden entries	10 min
Anatomy of a good entry	5 min

Lab Step 1 starts in class

Build yours

Step 1: Stakeholder Map
Step 2: First 3 golden entries
(Step 2 continues to 20 — homework)

Phase 1 Closing Week

You can ship. Now we measure.

PHASE 1 SO FAR

The platform is built

W1 — capstone framing + ADR draft
W2 — async pipeline, retries, store
W3 — FastAPI surface + minimal UI + tests
W4 — tool-calling, real streaming, real cost
5 weeks. One stable contract.



PHASE 1 ENDS THIS WEEK

The mental shift

Prompting patterns — name them
Eval — measure, don't just ship
Golden set — your ground truth
M1 milestone — defend Phase 1 work
DR #1 — first design review

Prompting Patterns

You've prompted a lot. The goal here is to name three patterns so they show up in design.

Prompting Patterns

Three patterns worth naming

1

Structured

The default

Role + Task + Context + Examples + Format.
On every prompt that matters.

2

Few-shot

When instructions alone fail

Two or three worked examples in the prompt. Beats elaborate instructions for new tasks.

3

Tool-aware

Sets up W19+ agents

When the LLM has tools, the prompt names them and gives use criteria.

Prompting Patterns • Pattern 1

Structured — Role + Task + Context + Examples + Format

```
# Every prompt that matters has five blocks
```

```
ROLE: You are an enterprise knowledge assistant.
```

```
TASK: Answer the user's question using only  
      the retrieved context. If the answer  
      isn't in the context, say so clearly.
```

```
CONTEXT: <retrieved chunks here>
```

```
EXAMPLES: <2-3 worked Q&A pairs>
```

```
FORMAT: Return JSON via the answer_question tool.
```

WHY EACH BLOCK

ROLE — anchors the model's persona

TASK — single clear instruction

CONTEXT — what to use (W6+: retrieved)

EXAMPLES — when format/style matters

FORMAT — how to return it

Skip any block when it doesn't apply —
but name what you're skipping.

Prompting Patterns • Pattern 2

Few-shot — show, don't tell

When instructions alone don't get the right format or style, two or three examples often do.

INSTRUCTIONS ONLY

Often vague results

```
Summarise this in  
the company's tone.
```

```
# What tone?  
# What length?  
# What focus?
```

WITH 2 EXAMPLES

Pattern is clear

```
Example 1:  
Input: ... → Output: ...
```

```
Example 2:  
Input: ... → Output: ...
```

```
Now summarise this:
```

Prompting Patterns • Pattern 3

Tool-aware — name the tools, give use criteria

When the LLM has tools (W4 onwards), the prompt should describe when to use each one — not just list them.

You have access to:

- `retrieve_docs(query: str) -> List[Document]`
Use when the user asks about company policies, processes, or anything specific to internal documents.
- `answer_question(content: str, confidence: float, sources)`
Use when you're ready to return the final answer.

Always retrieve before answering specific policy questions.

WHY THIS MATTERS

W4 — your ask_llm already uses tool-calling

W6+ — adds `retrieve_docs`

W13+ — multiple tools, model picks

W19+ — agents, where this matters most
The pattern is here to stay.

Why Unit Tests Don't Work for LLMs

`assert answer == 'expected'` is broken on day one. What replaces it.

The Eval Problem

Same prompt, three valid answers

Question: "What is the company's policy on remote work?" · Three runs of the same prompt:

Response 1

The company allows remote work up to 3 days per week, subject to manager approval.

Correct + concise

Response 2

Employees may work remotely for a maximum of three days each week, provided their manager has approved the arrangement.

Correct + verbose

Response 3

Yes, remote work is allowed.

Technically correct + missing the limit

assert answer == "expected" fails on ALL THREE. Including the correct ones.

The Eval Problem • The Fix

Evaluate properties, not strings

Stop asking 'is the output equal to X?'. Start asking 'does the output have the right properties?'

Is it accurate?

Does it answer the question correctly given the source material?

Is it grounded?

Is every factual claim traceable to the provided context — or honest about uncertainty?

Does it follow format?

JSON parses? Required fields filled? Length within bounds? Citations present?

A curated set of test cases + the properties each answer should have is a golden set.

The Eval Problem • W3 ↔ W5 Bridge

Testing and Evaluation are siblings

Different tools, different question. Both are how a real engineer keeps an LLM app honest.

TESTING • W3

Does the deterministic part work?

Tool: pytest, assertions, mocks

Question: did the code path behave?

Answers: yes / no

Run: on every commit, in CI

Best at: contracts, error paths, regressions

EVALUATION • W5

Are the LLM outputs any good?

Tool: golden set, LLM-as-judge, rubric

Question: do the outputs have the right properties?

Answers: score / distribution

Run: per prompt change, per model swap

Best at: quality drift, prompt iterations, model choice

Building a Golden Set

Twenty well-chosen questions become the ground truth for the rest of the programme.

Golden Set • Anatomy

Four fields per entry

```
{  
  "id": "g001",  
  "question": "What is the leave policy?",  
  "ideal_answer": "Employees get 20 days of paid leave per year, accrued month",  
  "notes": "Happy path. Source: HR-policy.pdf §3.1"  
}
```

WHY EACH FIELD

id — git-friendly stable reference (g001–g020)

question — exactly as a user would phrase it

ideal_answer — full text OR a properties dict if open-ended

notes — category, source doc, why it's interesting

Stored as .jsonl — one entry per line, easy to diff.

Golden Set • Three Samples • Enterprise Knowledge Assistant

Mix happy, harder, and edge

g001 • Happy path

What is the leave policy?

Easy retrieval. Tests the basic pipeline. 14 of your 20 entries.

g015 • Harder

How does the remote work policy interact with the team meeting requirement?

Multi-hop reasoning across two policy sections. 4 of your 20.

g019 • Edge — answer not in docs

What is the company's stance on cryptocurrency investing?

Should refuse gracefully, not hallucinate. 2 of your 20.

Golden Set • Size + Sourcing

Twenty is the floor — and quality beats quantity

SIZE GUIDANCE

20 minimum • 100+ for serious

20 — learn anything (this week)

50 — early signal you can trust

100+ — production-grade eval

Resist the temptation to write 50 mediocre questions.

20 great ones is more useful than 100 careless ones.

SOURCING — IN ORDER OF PREFERENCE

Real users beat made-up

BEST — questions from actual users / Slack / support tickets

GOOD — questions a domain expert imagines a user asking

OK — you sit at the keyboard and write some

AVOID — questions designed to make your system look good

Bias toward the hard ones. Easy questions don't teach you anything.

Day 1 hands-on • Lab Steps 1 + 2

Stakeholder Map + first golden entries

STEP 1 • 20 MIN

Stakeholder Map

- 1 Open the worksheet template
- 2 Fill 5 prompts: user, sponsors, before, after, top 3 metrics
- 3 Commit as docs/adr/0001-stakeholder-map.md

→ This LOCKS the use case for MI.

STEP 2 • 45 MIN

20-sample golden set

- 1 Write 3 entries together in class
- 2 14 happy-path • 4 harder • 2 edges
- 3 Save as data/golden_set.jsonl + notes doc

→ Step 2 finishes tonight as homework.

Day 1 • Wrap

Three things you learned

Prompting has patterns

Structured • Few-shot • Tool-aware. Name them; they show up in design discussions.

Unit tests don't work for LLMs

Same prompt → three valid answers.
Evaluate properties, not strings.

Golden sets are infrastructure

Twenty well-chosen questions become every later week's KPI baseline.

TOMORROW • DAY 2

LLM-as-judge. Pairwise comparison. Critic-Creator loop. M1 deliverable walk-through.
By Friday: ADR v1 (Locked) committed. DR #1 1-pager ready. Phase 1 defensible.

Day 2 • Judge, Refine, Defend

Today's Outline

Concepts

What you'll learn

LLM-as-judge + pairwise	25 min
Critic-Creator loop	15 min
M1 + DR #1 walkthrough	15 min

Live demos

Watch it built

Judge prompt for one entry	10 min
Pairwise — flip the position	5 min
Critic-Creator round-by-round	10 min

Lab Steps 3–6

Build yours

- Step 3: judge over 20 entries
- Step 4: pairwise — two prompts
- Step 5: Critic-Creator on one Q
- Step 6: ADR v1 Locked + DR #1 1-pager

LLM-as-judge, pairwise, biases

Three eval techniques — what each is good for, what each gets wrong.

LLM-as-Judge • Rubric Scoring

Rubric scoring — define dimensions, ask the judge

```
RUBRIC = """
Score each dimension 1-4:

ACCURACY      – Is the answer factually correct given
                the source documents?

GROUNDEDNESS  – Does every claim trace to the context,
                or honestly admit uncertainty?

FORMAT        – Required fields present? Length right?
                Citations included?
"""

# Judge prompt: rubric + question + candidate answer
# → judge returns {accuracy: 3, groundedness: 4, format: 4}
```

WHY 1-4 (NOT 1-10)

Coarse scales are more reliable. Judges struggle to distinguish 7 from 8 on a 1-10.
4 levels map cleanly to: Poor / OK / Good / Excellent.
Same scale as the Design Review Rubric — you'll see this again on Friday.

LLM-as-Judge • Pairwise

Pairwise — easier to judge, less biased

Instead of "score this 1-4", ask the judge: "which of these two is better, and why?"

ABSOLUTE SCORING

Score answer A: 3

Judges anchor inconsistently — what's a 3 vs a 4?

Drifts across runs of the same answer.

Useful as a coarse signal — "is this passing?"

Bad at small differences.

PAIRWISE • PREFERRED

Pick: A or B + why

Judges are much better at relative judgments.

Direct signal — "which prompt won more questions?"

Stable across runs.

Catch small differences that absolute scoring misses.

LLM-as-Judge • Gotchas

Three biases that will lie to you

Length bias

Longer answers win — even when wrong

Judges interpret length as effort. A confident wrong long answer often beats a correct short one.

Position bias

First option wins — when nothing else differs

Show A then B, A wins more than 50% of the time. Show B then A, B wins. Pure ordering effect.

Self bias

Judge prefers its own model's style

GPT-4 judging GPT-4 outputs vs Claude outputs leans GPT-4. Use a different judge, or be explicit.

LLM-as-Judge • Mitigations

How to fight back — three practical moves

1

Flip position randomly

In pairwise, half your runs show A first, half show B first. The bias averages out across the set.

2

Use a strong judge

GPT-4o or Claude Sonnet — not gpt-4o-mini, not llama 3B. Cheap judges miss differences. A weak judge is worse than no judge.

3

Spot-check 10%

Read 2 of every 20 judgements yourself. If the judge agrees with you 90%+ of the time, trust it. If not, fix the rubric.

LLM-as-Judge • Live Demo

Run the judge — see the score and the reasoning

On screen — score one golden entry on three dimensions. Then run pairwise on two prompts.

WHAT YOU'LL SEE

- | | | |
|---|---------------------------------|---|
| 1 | Define the rubric prompt | 3 dimensions, 1-4 scale, ask for reasoning |
| 2 | Call the judge once | gpt-4o via tool-calling → structured output |
| 3 | Read the reasoning | Does the judge agree with you? Spot the disagreement |
| 4 | Flip pairwise position | Same two answers, A-then-B vs B-then-A — note any drift |

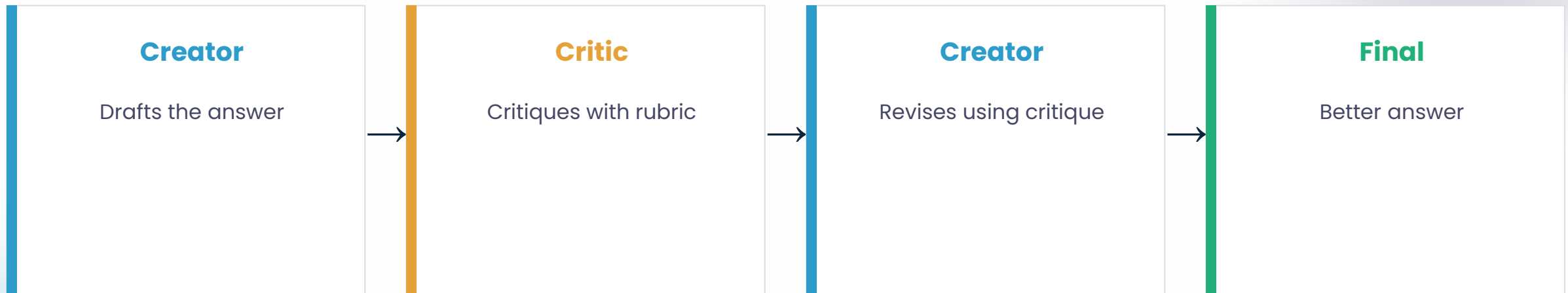
→ Detailed walkthrough in Lab Guide Step 3. Reference code at `<cohort-repo>/week5/reference/judge.py`.

Critic-Creator Loop

The first systematic prompting tool. The shape of every reflection pattern you'll meet later.

Critic-Creator

Two prompts, one loop, materially better answers



TWO TO THREE ROUNDS

Each round, the Critic catches something the Creator missed. After 2-3 rounds, the answer is materially better — and you can stop on convergence.

Critic-Creator • Implementation

Thirty lines of Python

```
# Round-by-round loop
answer = await creator(question)

for round_n in range(MAX_ROUNDS):
    critique = await critic(question, answer, RUBRIC)

    if critique.is_good_enough:
        break

    answer = await creator.revise(question, answer, critique)

# Each round, the critic returns:
#   - score per rubric dimension
#   - list of specific problems with this draft
#   - is_good_enough: bool – stop the loop on convergence
```

WHAT TO NOTICE

Two prompts, not one model.

The critic gets the same rubric your judge uses.

Convergence check stops the loop early
— saves tokens.

Each round is a real API call.

→ Full code in lab Step 5.

Critic-Creator • Why This Matters

You'll see this pattern again — and again

Two-prompt iteration is the shape of every reflection/self-refinement pattern in modern LLM apps.

W11

Eval frameworks

RAGAS uses LLM-as-judge under the hood — same shape.

W17

Reflection loops

Multi-step agents that critique their own plans.

W22

Plan-and-revise

Long-running agent tasks that revise mid-execution.

W28

Production evaluation

Continuous judge runs in production, gated deployments.

Day 2 hands-on • Lab Steps 3 + 4 + 5

Judge, pairwise, Critic-Creator — on your golden set

Step 3

45 min

Judge over 20 entries

Run `src/eval/judge.py` against every golden entry via your `W4 /ask` endpoint. Score on 3 dimensions. Persist to the `eval_runs` table. Aggregate stats in `docs/eval-run-001.md` — your M1 baseline.

Step 4

30 min

Pairwise — two prompts

Write a v2 of your answer-generation prompt (one specific change). Run both v1 and v2 via `/ask` on the golden set. Pairwise the judge with random position-flip. Document the winner in `docs/prompt-pairwise-001.md`.

Step 5

20 min

Critic-Creator on one Q

Pick one of your harder golden questions (`g015-g020`). Run Critic-Creator for 2–3 rounds. Save the round-by-round trace in `docs/critic-creator-trace.md`. Note where the critic caught real problems.

M1 + Design Review #1

Six deliverables. Five rubric dimensions. Fifteen minutes per learner. You've done the work.

M1 - DELIVERABLE CHECKLIST

Six items — all six required

a	ADR v1 (Locked)	docs/adr/0001-capstone-framing.md — locked use case, architecture, 2-3 KPIs
b	FastAPI service	/ask + /health (from W3) with W4 internals upgrade landed
c	Minimal UI	Streamlit from W3 — connects, sends question, shows answer
d	Stakeholder Map	docs/adr/0001-stakeholder-map.md from Lab Step 1
e	Golden set + judge baseline	data/golden_set.jsonl (20 entries) + docs/eval-run-001.md
f	DR #1 1-pager	docs/dr/dr1-summary.md — what you built, what you measured, what's next

Lab Step 6 is where you finalise (a) and (f). Items (b)-(e) already exist from W3, W4, and earlier lab steps.

DR #1 - THE RUBRIC

Five dimensions × four levels — pre-read in the workbook

Dimension	Level 1 — Poor	Level 3 — Good	Level 4 — Excellent
Problem clarity	User unclear	Use case named, sponsors listed	Stakeholder map specific + measurable
Architecture	Hand-wavy	Diagram + ADR exists	Diagram + ADR + tradeoffs articulated
Evidence	Demo only	Demo + tests passing	Demo + tests + eval numbers
Decisions	Implicit	Recorded in ADR	ADR + reasons + counterexamples
Defensibility	Defensive	Answers most challenges	Welcomes challenges; updates ADR live

DR #1 isn't a pass/fail — it's the trajectory marker. Passing bars are at DR #3, #5, #7.

DR #1 - FORMAT

15 minutes per learner — scheduled separately



Peers help, don't attack. The rubric tracks trajectory across seven DRs — not a single test.

Lab Step 6 - THE M1 PUSH

ADR v1 (Locked) + DR #1 1-pager + peer dry-run

1

Extend ADR 0001 to v1 (Locked)

Pull in chosen model + cost budget + KPIs + locked use case + eval baseline. Mark as Defended-at-DR#1.

2

Write the DR #1 1-pager

docs/dr/dr1-summary.md — one page. What you built, what you measured, top 3 things to discuss.

3

Peer dry-run

Find one cohort peer. Present in 5 min. Have them ask 3 hard questions. Note what you couldn't answer cleanly.

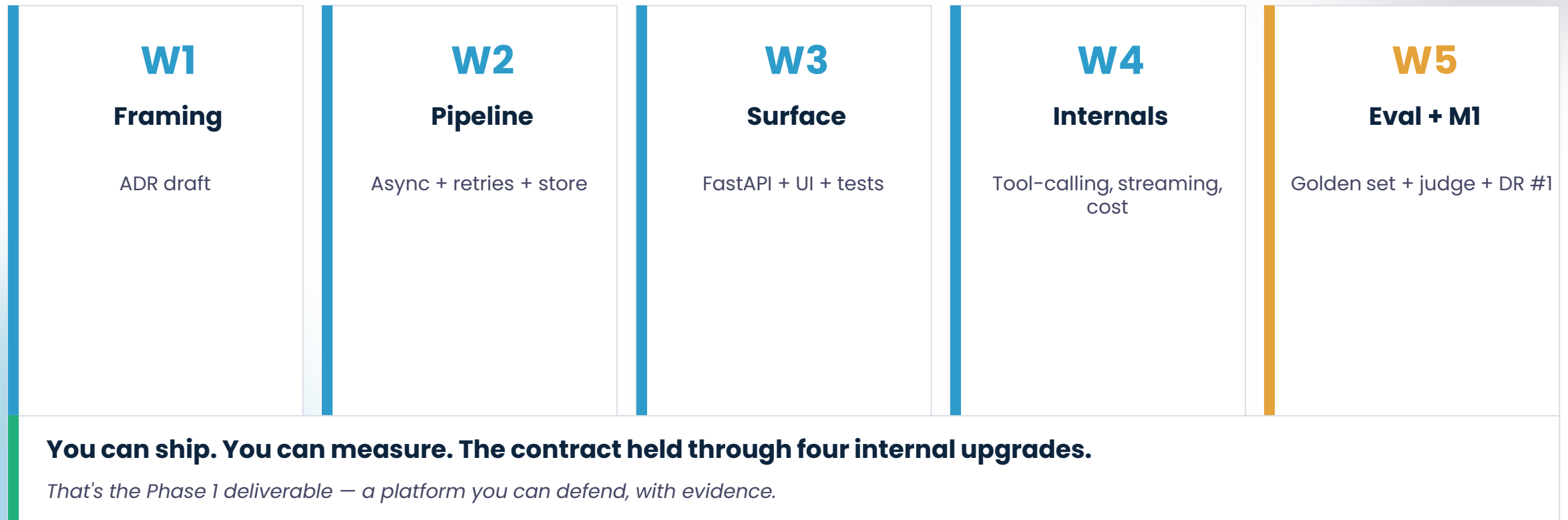
4

Commit M1

All six M1 items in one PR titled 'M1 — Phase 1 deliverable'. Tag your DR #1 reviewers.

Phase 1 - Retrospective

Five weeks. One platform you can defend.



Phase 2 - Starts Next Week

Phase 2 — RAG (W6–W12)

You have a platform that takes a question and returns an answer. Phase 2 makes it answer from documents.

W6	Naive RAG from Scratch	Embeddings + a vector store + the simplest possible RAG.
W7	Chunking strategies	Document splitting that doesn't murder context.
W8	Retrieval quality	Recall@k, MRR — measured against your W5 golden set.
W9	Hybrid search	Dense + sparse + reranking.
W10	Query transformation	HyDE, query expansion, sub-questions.
W11	Eval frameworks	RAGAS — formalising what you built in W5.
W12	Phase 2 milestone (M2)	DR #2. Production RAG defended.

Week 5 - Wrap

Phase 1 is yours to defend.

Five things you can now do — and the DR #1 mindset.

1

Name three prompting patterns

2

Build a golden set

3

Run LLM-as-judge

4

Pairwise + Critic-Creator

5

Defend a milestone

DR #1 · MINDSET

Show what you have. Defend your decisions.

Welcome the challenge — every question your peers ask is one you won't get blindsided by later.

Q & A

Thank you